EXAM-DAY CHEAT SHEET - Python (print this, ~5 pages)

Version 2: now covers EVERY item on the instructor's practical-assessment checklist. The code blocks form ONE continuous, tested script (pandas -> merge -> charts -> SQLite -> CSV export -> PyTorch regression). Use each block as a set of INDEPENDENT operations: copy only the lines you need and change the names. Explanations: Topics/T1-T9 and Paper_Solutions/P1.

Before you type: exact names? (file, DB, table, columns) - look at the data: `shape`, `head()`, `isna().sum()`, `duplicated().sum()`, `dtypes` - run after every step - print every answer.

A. Imports and loading

```
import sqlite3, os, csv
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch, torch.nn as nn, torch.optim as optim

df = pd.read_csv("sales_data.csv")                # load CSV
print(df.shape)                                   # (rows, columns)
print(df.head()); print(df.dtypes); print(df.isna().sum())
```

B. Duplicates, cleaning, merge, filter, save

```
print(df.duplicated().sum())                      # how many repeated rows
print(df[df.duplicated(keep=False)].head(4))      # SEE them (all copies)
before = df.shape
df = df.drop_duplicates()                         # keep first copy
# df.drop_duplicates(subset=["Date", "Product"], keep="first")  # by columns
print("shape", before, "->", df.shape)

df["Product"] = df["Product"].fillna("Unknown")                  # text blank
df["Price"] = df["Price"].fillna(df["Total"] / df["Quantity"])   # recover
df["Price"] = df["Price"].fillna(0)                              # fallback
df["Total"] = df["Quantity"] * df["Price"]                       # fix Total
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)           # dd/mm/yyyy
# other fills: .fillna(df["x"].mean())  .median()  .mode()[0]   dropna(subset=[..])

# MERGE two DataFrames on a common column
cats = pd.DataFrame({"Product": ["Chocolate", "Biscuits", "Sweets", "Water",
                                 "Orange Juice"],
                     "Category": ["Snack", "Snack", "Snack", "Drink", "Drink"]})
merged = df.merge(cats, on="Product", how="left")   # inner | left | right | outer
print(df.shape, cats.shape, merged.shape)           # CHECK rows did not blow up
# different key names: a.merge(b, left_on="id", right_on="cust_id")
# see unmatched rows: how="outer", indicator=True  -> column _merge

# SELECT / FILTER
d23 = merged[merged["Date"].dt.year == 2023]                        # one condition
big = merged[(merged["Total"] > 100) & (merged["Category"] == "Snack")]  # & | ~  + brackets
two_cols = merged[["Product", "Total"]]                             # choose columns
top = merged.sort_values("Total", ascending=False).head(5)          # sort + first 5
# merged.loc[merged["Total"] > 100, ["Date", "Total"]]    isin([..])   between(a, b)

# SAVE to CSV
big.to_csv("big_sales.csv", index=False)            # index=False: no 0,1,2 column
print(pd.read_csv("big_sales.csv").shape)           # read it back to check
```

C. Matplotlib (every plot: NEW figure + title + xlabel + ylabel)

```
qty = merged.groupby("Product")["Quantity"].sum()                       # pile + add
month = d23.groupby(d23["Date"].dt.month)["Total"].sum()                # 2023 by month

plt.figure(figsize=(8, 5))                          # NEW paper for EVERY chart
plt.bar(qty.index, qty.values)                      # BAR
plt.title("Quantity per product"); plt.xlabel("Product"); plt.ylabel("Quantity")
plt.xticks(rotation=45); plt.tight_layout(); plt.show()

plt.figure(); plt.plot(month.index, month.values, marker="o")           # LINE
plt.title("Sales 2023"); plt.xlabel("Month"); plt.ylabel("Total"); plt.show()

plt.figure(); plt.scatter(merged["Quantity"], merged["Total"])          # SCATTER
plt.title("Quantity vs Total"); plt.xlabel("Quantity"); plt.ylabel("Total"); plt.show()

names = list(qty.index)                                                 # BOX PLOT
groups = [merged.loc[merged["Product"] == p, "Total"] for p in names]   # 1 list per box
plt.figure(); plt.boxplot(groups)                   # box = middle 50%, line = median
plt.xticks(range(1, len(names) + 1), names, rotation=45)   # name the boxes (works in every version)
plt.title("Total by product"); plt.xlabel("Product"); plt.ylabel("Total"); plt.tight_layout(); plt.show()
# one-liner: merged.boxplot(column="Total", by="Product")      histogram: plt.hist(v, bins=5)
# pie: plt.pie(v, labels=l, autopct="%1.1f%%")     save instead of show: plt.savefig("a.png")
```

D. SQLite: DataFrame -> table, SELECT/WHERE/ORDER BY, UPDATE, export to CSV

```
conn = sqlite3.connect("SalesDB.db"); cur = conn.cursor()               # connect
cur.execute("""CREATE TABLE IF NOT EXISTS Sales (
                   Date TEXT, Product TEXT, Quantity INTEGER,
                   Price REAL, Total REAL,
                   UNIQUE (Date, Product, Quantity, Price, Total))""")
out = df.copy(); out["Date"] = out["Date"].dt.strftime("%Y-%m-%d")      # dates -> text
for _, r in out.iterrows():                                             # insert, no duplicates
    cur.execute("INSERT OR IGNORE INTO Sales VALUES (?, ?, ?, ?, ?)",
                (r["Date"], r["Product"], int(r["Quantity"]),
                 float(r["Price"]), float(r["Total"])))
conn.commit()                                       # save! (no commit = lost)

# store a whole DataFrame as a table (one line)
out.to_sql("SalesCopy", conn, if_exists="replace", index=False)         # fail | replace | append

# SELECT + WHERE + ORDER BY with ? parameters
cur.execute("""SELECT Product, SUM(Quantity) AS q FROM Sales
               WHERE strftime('%Y', Date) = ?
               GROUP BY Product ORDER BY q DESC""", ("2023",))
for product, q in cur.fetchall():
    print(product, q)
cur.execute("SELECT * FROM Sales WHERE Product = ? AND Total > ? ORDER BY Date DESC",
            ("Chocolate", 40))                      # ONE value needs a comma: ("x",)
rows = cur.fetchall()
cols = [c[0] for c in cur.description]              # column names - grab NOW (after an
                                                    # UPDATE, description is None)
# UPDATE (always WHERE! check rowcount, then commit)
cur.execute("UPDATE SalesCopy SET Price = ? WHERE Product = ?", (11.0, "Chocolate"))
print(cur.rowcount, "rows updated")                 # how many rows changed
conn.commit()
# arithmetic: UPDATE T SET Price = Price * 1.1 WHERE ...     DELETE FROM T WHERE ...

# EXPORT query results to CSV (two ways)
pd.DataFrame(rows, columns=cols).to_csv("query_result.csv", index=False)
pd.read_sql_query("SELECT * FROM Sales WHERE Product = ?", conn,
                  params=("Water",)).to_csv("water.csv", index=False)
conn.close()
```

E. CSV file: create-or-append, input loop, numpy summary

```
FILE = "heights.csv"
if not os.path.exists(FILE):
    with open(FILE, "w", newline="") as f:
        csv.writer(f).writerow(["Name", "Height"])
    print("heights.csv created")
else:
    print("heights.csv exists - appending")

for name, h in [("Amy", "165"), ("Ben", "172"), ("q", "")]:   # replace with input()
    if name.lower() == "q":                    # name = input("Name (q to quit): ")
        break
    try:
        height = int(h)                        # h = input("Height (cm): ")
    except ValueError:
        print("Height must be a whole number"); continue
    with open(FILE, "a", newline="") as f:     # a = append
        csv.writer(f).writerow([name, height])

h = pd.read_csv(FILE)["Height"].to_numpy()
text = (f"Records: {len(h)}\nAverage: {np.mean(h):.1f}\n"
        f"Min: {np.min(h)}\nMax: {np.max(h)}\n")
print(text)
with open("height_summary.txt", "w") as f:
    f.write(text)
plt.figure(); plt.hist(h, bins=5)
plt.title("Height Distribution"); plt.xlabel("Height (cm)"); plt.ylabel("Frequency")
plt.show()
```

F. PyTorch REGRESSION (the instructor's recipe: y = X @ w + b + noise)

```
torch.manual_seed(42)                          # FIRST - before data AND before the model
N, F = 200, 3                                  # 200 samples, 3 features
X = torch.randn(N, F)                          # X shape (N, F)
w = torch.tensor([[2.0], [-1.0], [0.5]])       # true weights, shape (F, 1)
b = 0.7                                        # true bias
y = X @ w + b + 0.1 * torch.randn(N, 1)        # (N,F)@(F,1) -> (N,1)  + noise
print(X.shape, y.shape)                        # torch.Size([200, 3]) torch.Size([200, 1])

class RegNet(nn.Module):
    def __init__(self):
        super().__init__()                     # FIRST line of __init__, always
        self.fc1 = nn.Linear(F, 8)             # F inputs -> 8 hidden neurons
        self.fc2 = nn.Linear(8, 1)             # 8 -> 1 output number
    def forward(self, x):
        return self.fc2(torch.relu(self.fc1(x)))   # NO sigmoid: output is any real number

model = RegNet()
criterion = nn.MSELoss()                       # continuous target -> MSE
optimizer = optim.Adam(model.parameters(), lr=0.01)    # or optim.SGD(..., lr=0.01)
losses = []
for epoch in range(200):
    optimizer.zero_grad()                      # 1 wipe old gradients
    loss = criterion(model(X), y)              # 2+3 forward pass, loss
    loss.backward()                            # 4 backpropagation
    optimizer.step()                           # 5 update weights
    losses.append(loss.item())                 # record a plain number
print(f"loss {losses[0]:.3f} -> {losses[-1]:.4f}")

plt.figure(); plt.plot(losses)
plt.title("Training loss"); plt.xlabel("Epoch"); plt.ylabel("MSE loss"); plt.show()
with torch.no_grad():                          # check: predictions vs truth
    print(torch.cat([model(X[:3]), y[:3]], dim=1))
# classification instead: end with torch.sigmoid, loss nn.BCELoss(), y = 0./1. shape (N,1)
# 2024 paper: Linear(10,5) relu Linear(5,3) relu Linear(3,1) sigmoid, MSELoss, SGD lr=0.01
```

G. Python reminders

```
f"{x:.2f}" 2 decimals    {x:>8} right-align   {name:<10} left-align
int(input("n: "))        try: ... except ValueError: ...      while True: ... break
for i, v in enumerate(lst):      dict.get(k, 0)      sorted(d.items(), key=lambda kv: kv[1], reverse=True)
class A: def __init__(self, x): self.x = x      def __str__(self): return f"A({self.x})"
and / or / not      == (compare) vs = (assign)       lst[1:4]  s[::-1]   len(x)
```

H. Error -> cause -> fix

You see	Cause	Fix
`ValueError: time data ... doesn't match`	day/month order	`dayfirst=True` or `format=`
`Can only use .dt accessor with datetimelike values`	Date still text	convert with `to_datetime` first
`KeyError: 'zzz'` (merge or select)	wrong column name / spaces	`print(df.columns)`
`ValueError: You are trying to merge on object/str and int64 columns`	key types differ	`a["k"] = a["k"].astype(int)`
merged table has far MORE rows than expected	duplicate keys on both sides	`drop_duplicates` on the key first, compare `shape`
`ValueError: x and y must be the same size`	scatter lists differ in length	check `len()` of both
`OperationalError: no such table` / `no such column: zzz`	queried before CREATE / typo / wrong folder	create first; check names and cwd
`UNIQUE constraint failed`	duplicate with plain INSERT	`INSERT OR IGNORE`
`ProgrammingError: Incorrect number of bindings supplied`	`?` count != tuple length, or `("x")` instead of `("x",)`	match counts; add the comma
BLOB junk / `Error binding parameter`	numpy int / Timestamp into SQLite	`int()`, `float()`, `strftime`
table unchanged after UPDATE	forgot `commit()` (or WHERE matched nothing: `rowcount` 0)	`conn.commit()`; print `cur.rowcount`
`mat1 and mat2 shapes cannot be multiplied`	layer sizes do not chain	out of one = in of next
no error but `y` has shape `(N, N)` (or a loss warning)	`w` was `(F,)` not `(F,1)`: `(N,)` + noise `(N,1)` broadcasts silently	`w.reshape(F, 1)`; always print `X.shape, y.shape`
results differ every run	seed missing or set too late	`torch.manual_seed(42)` on the first line
`AttributeError: cannot assign module before Module.__init__()`	missing `super().__init__()`	add it first
`FileNotFoundError`	CSV not beside the script	check `os.getcwd()`

I. Last 10 minutes

- [ ] delete `.db`, run the **whole file** from a clean start - no errors
- [ ] every chart: title + both axis labels - every answer printed with a label
- [ ] shapes printed (`df.shape`, `X.shape`, `y.shape`); seed on the first torch line
- [ ] comments quote the question; names exact; file named as the paper says; saved and uploaded